LAPORAN AKHIR PROYEK KELOMPOK

Vibe Coding Generative AI from Scratch

Pix2Pix: Translasi Label Map menjadi Foto Fasad Bangunan

Mata Kuliah: Generative AI

Kelas: BS1IF-47-GAB03

Kelompok: 9



Anggota:

Zafira Nur Kamila (103012330195): Prompt Engineer, Model & Training

Safira (103012330038): Data Pipeline & Preprocessing Engineer

Celsyka Zahra Sabila (103012300144): Lead Documentarian & Error Handler

Repositori: https://github.com/zafiranurkamila/pix2pix-scratch

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM

BANDUNG

2026

DAFTAR ISI

BAB 1. PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Tujuan 3
1.3 Ruang Lingkup dan Batasan 3
1.4 Metode Vibe Coding dan Pembagian Tugas 3
BAB 2. LANDASAN TEORI **5**
2.1 Generative Adversarial Network (GAN) 5
2.2 Conditional GAN 5
2.3 Pix2Pix 5
2.4 Generator U-Net dan Skip Connection 5
2.5 Discriminator PatchGAN 5
2.6 Fungsi Loss 6
BAB 3. METODOLOGI **7**
3.1 Dataset (CMP Facades) 7
3.2 Preprocessing dan Augmentasi 7
3.3 Arsitektur Model 8
3.4 Fungsi Loss dan Training 10
3.5 Lingkungan Eksperimen 11
3.6 Alur Kerja Tim dan Git 11
BAB 4. EKSPERIMEN DAN HASIL **12**
4.1 Hasil Training Baseline 12
4.2 Eksperimen Tuning 13
4.3 Hasil Visual 15
4.4 Evaluasi Kuantitatif 17
BAB 5. LOG INTERAKSI LLM DAN PENANGANAN ERROR **18**
5.1 Ringkasan Sesi 18
5.2 Rekap Error dan Solusi 19
5.3 Pola Prompting yang Efektif 20
BAB 6. KONTRIBUSI DAN KERJA SAMA TIM **23**
6.1 Kontribusi Anggota 23
6.2 Cara Integrasi 23
6.3 Kendala Kerja Sama dan Cara Mengatasinya 23
BAB 7. KESIMPULAN DAN SARAN **24**
7.1 Kesimpulan 24
7.2 Keterbatasan 24
7.3 Saran 24
DAFTAR PUSTAKA **25**
LAMPIRAN **26**

ABSTRAK

Proyek ini membangun model Pix2Pix, yaitu conditional GAN untuk translasi gambar berpasangan, dari nol menggunakan PyTorch dengan metode vibe coding bersama Large Language Model (LLM). Model diterapkan pada dataset CMP Facades untuk mengubah label map fasad bangunan menjadi foto fasad. Arsitekturnya terdiri atas generator U-Net (sekitar 54,4 juta parameter) dan discriminator PatchGAN 70×70 (sekitar 2,8 juta parameter) yang dilatih dengan gabungan loss adversarial (BCE) dan L1. Lima konfigurasi dilatih masing-masing 200 epoch, lalu dievaluasi pada 106 gambar test dengan metrik L1, PSNR, SSIM, dan ketajaman. Hasilnya menunjukkan bahwa semua konfigurasi mampu mengikuti struktur label map. Tanpa loss adversarial, hasil gambar sangat buram meskipun skor kemiripannya tertinggi. Konfigurasi dengan batch size 4 memberikan hasil terbaik di antara model GAN pada sebagian besar metrik dan sekaligus 2,4 kali lebih cepat dilatih. Kerja kelompok didokumentasikan melalui Git dan log interaksi LLM.

Kata kunci: Pix2Pix, conditional GAN, U-Net, PatchGAN, CMP Facades, vibe coding.

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Image-to-image translation adalah tugas mengubah sebuah gambar dari satu domain ke domain lain dengan tetap mempertahankan struktur isinya, misalnya mengubah sketsa menjadi foto, peta menjadi citra satelit, atau peta label bangunan menjadi foto fasad. Tugas ini bermanfaat di bidang arsitektur, desain, dan pembuatan konten, karena pengguna cukup menggambar struktur kasar lalu model melengkapi detail visualnya.

Pix2Pix (Isola dkk., 2017) adalah salah satu pendekatan yang paling dikenal untuk tugas ini ketika tersedia data berpasangan. Model ini berupa conditional GAN dengan generator U-Net dan discriminator PatchGAN. Kelompok kami memilih Pix2Pix karena arsitekturnya jelas dan bertahap, mencakup konsep-konsep penting generative AI (dua jaringan yang saling berlawanan, skip connection, dan gabungan loss), serta dapat dilatih pada satu GPU dengan dataset berukuran kecil.

1.2 Tujuan

- Membangun Pix2Pix dari nol (from scratch) dengan PyTorch menggunakan metode vibe coding bersama LLM, tanpa menyalin kode dari tutorial.
- Melatih model pada dataset CMP Facades untuk menerjemahkan label map menjadi foto fasad.
- Membandingkan beberapa konfigurasi hyperparameter dan mengevaluasi hasilnya secara kuantitatif maupun visual.
- Mendokumentasikan seluruh interaksi dengan LLM, termasuk penanganan error, serta kerja sama tim melalui Git.

1.3 Ruang Lingkup dan Batasan

- Arah translasi: label map → foto fasad (bukan sebaliknya).
- Ukuran gambar 256 × 256 piksel, RGB (3 channel).
- Framework PyTorch. Seluruh training dijalankan di PC lokal dengan GPU NVIDIA GeForce RTX 5060 Ti (16 GB VRAM).
- Model dibangun dan dilatih dari nol, tanpa bobot pretrained.
- Setiap konfigurasi dilatih 200 epoch dengan seed 42 (satu seed per konfigurasi).
- Evaluasi dilakukan pada split test (106 gambar).

1.4 Metode Vibe Coding dan Pembagian Tugas

Vibe coding adalah cara membangun perangkat lunak dengan berkolaborasi aktif bersama LLM, manusia yang menentukan arah, kebutuhan, dan batasan, sedangkan LLM menghasilkan kode yang kemudian diuji dan diperbaiki melalui dialog. Pada proyek ini, kelompok lebih dulu menetapkan kontrak teknis (`KONTRAK.md`) yang memuat keputusan arsitektur, format tensor, dan signature kelas. Dengan begitu, bagian yang dikerjakan tiap

anggota bersama LLM dapat digabung tanpa bentrok. Setiap sesi dengan LLM dicatat dalam log interaksi (Bab 5).

Anggota	Peran	Tanggung jawab dan file	Branch
Zafira	Prompt Engineer, Model & Training	Strategi arsitektur dan prompt; `src/models/`, `config.py`, `losses.py`, `train.py`, `utils.py`, `metrics.py`, `evaluate.py`, `compare_runs.py`; eksperimen tuning	`feat/model`, `feat/training`
Fira	Data Pipeline & Preprocessing Engineer	Dataset berpasangan, preprocessing, augmentasi, DataLoader, visualisasi; `src/dataset.py`	`feat/dataset`
Ce	Lead Documentarian & Error Handler	Kompilasi log interaksi LLM, pengelolaan repo dan PR, laporan akhir; docs/	`docs/laporan`

BAB 2. LANDASAN TEORI

2.1 Generative Adversarial Network (GAN)

GAN (Goodfellow dkk., 2014) terdiri atas dua jaringan saraf yang dilatih saling berlawanan. Generator berusaha menghasilkan sampel palsu yang tampak nyata, sedangkan discriminator berusaha membedakan sampel asli dari sampel palsu. Keduanya dilatih bergantian dalam permainan minimax. Setiap kali discriminator menjadi lebih pandai, generator terdorong menghasilkan sampel yang lebih meyakinkan.

2.2 Conditional GAN

Pada conditional GAN, generator dan discriminator sama-sama diberi informasi tambahan sebagai syarat (kondisi). Pada Pix2Pix, kondisi tersebut adalah gambar input, yaitu label map. Generator membuat foto berdasarkan label map, dan discriminator menilai apakah sebuah foto layak menjadi pasangan dari label map tertentu, bukan sekadar apakah foto itu tampak asli.

2.3 Pix2Pix

Pix2Pix (Isola dkk., 2017) adalah kerangka umum untuk translasi gambar ketika tersedia pasangan gambar input dan target. Pix2Pix memakai generator U-Net dan discriminator PatchGAN, dilatih dengan gabungan loss adversarial dan loss L1 terhadap gambar target. Loss adversarial mendorong hasil yang tajam dan realistis, sedangkan loss L1 menjaga hasil tetap dekat dengan target.

2.4 Generator U-Net dan Skip Connection

U-Net (Ronneberger dkk., 2015) awalnya dikembangkan untuk segmentasi citra biomedis. Arsitekturnya berbentuk encoder–decoder simetris: encoder memperkecil ukuran spasial sambil memperbanyak channel, lalu decoder memperbesarnya kembali. Pada Pix2Pix, encoder melakukan 8 kali downsampling sehingga gambar 256 × 256 menjadi 1 × 1. Itu sebabnya ukuran gambar harus kelipatan 256 (proyek ini memakai 256), karena ukuran lain membuat shape skip connection tidak cocok.

Skip connection menyambungkan output tiap blok encoder langsung ke blok decoder yang ukurannya sama. Tanpa jalur ini, seluruh informasi posisi harus melewati bottleneck 1 × 1 sehingga banyak detail struktur yang hilang.

2.5 Discriminator PatchGAN

PatchGAN tidak menilai gambar secara utuh, tetapi menilai tiap potongan kecil (patch) gambar. Pada konfigurasi 70 × 70, setiap nilai pada output menilai asli atau palsu untuk satu patch berukuran sekitar 70 × 70 piksel. Untuk input 256 × 256, outputnya berupa grid 30 × 30. Pendekatan ini membuat discriminator fokus pada tekstur dan detail lokal, memiliki parameter lebih sedikit, dan dapat diterapkan pada gambar berukuran berapa pun.

2.6 Fungsi Loss

Loss Pix2Pix merupakan gabungan dua suku. Suku adversarial memakai *binary cross entropy* (BCE) pada keluaran discriminator. Karena keluaran discriminator berupa logits tanpa sigmoid, BCE yang dipakai adalah `BCEWithLogitsLoss`, yang sudah menggabungkan sigmoid dan BCE secara stabil secara numerik. Suku kedua adalah loss L1, yaitu rata-rata selisih absolut piksel antara gambar hasil dan gambar target, dengan bobot λ. Rumus yang dipakai pada proyek ini dijelaskan pada subbab 3.4.

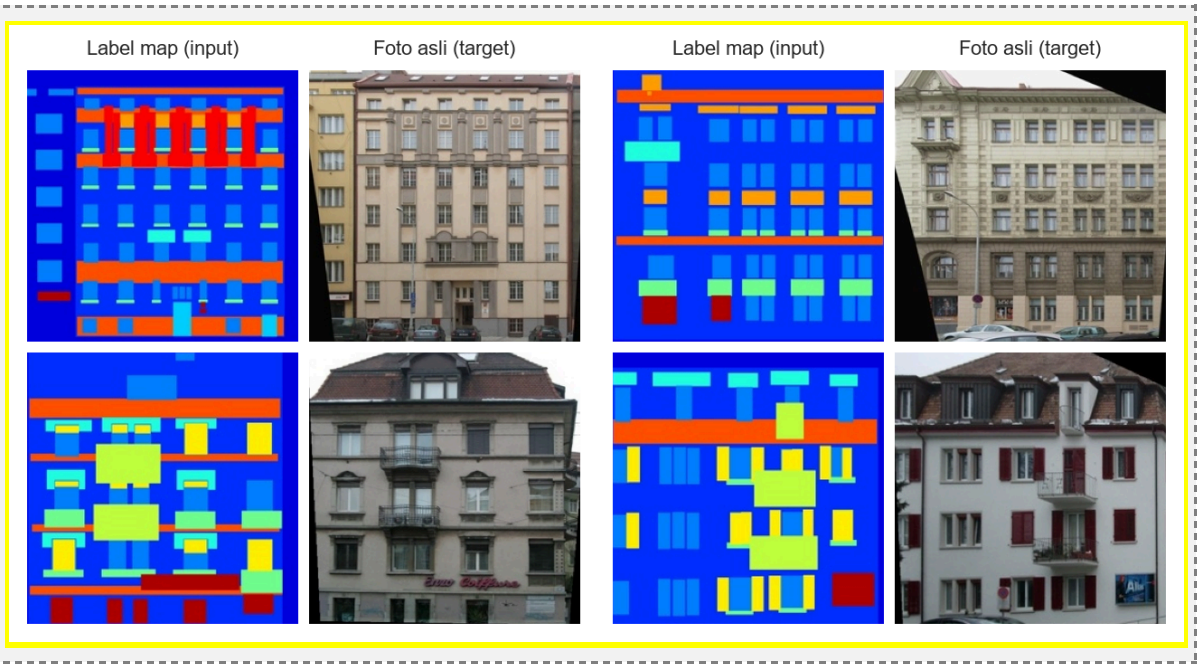
BAB 3. METODOLOGI

3.1 Dataset (CMP Facades)

Dataset yang digunakan adalah CMP Facades dalam versi dataset Pix2Pix, yang terbagi atas split `train`, `val`, dan `test`. Setiap berkas berisi dua gambar yang digabung berdampingan (kiri | kanan): satu berupa label map berwarna, dan satu lagi berupa foto asli fasad bangunan. Arah translasi yang dipakai adalah label map → foto.

Split	Jumlah gambar	Keterangan
train	400	Dilatih dengan augmentasi
val	100	Dipakai untuk memantau `val_L1` tiap epoch
test	106	Dipakai untuk evaluasi akhir

Sumber dataset: berkas facades.tar.gz dari halaman dataset resmi Pix2Pix (http://efrosgans.eecs.berkeley.edu/pix2pix/datasets/facades.tar.gz), yaitu versi siap pakai dari CMP Facade Database (Tyleček & Šára, 2013). Hasil pengecekan manual posisi gambar: sisi kiri berisi foto asli dan sisi kanan berisi label map. Pengecekan ini diwajibkan oleh kontrak teknis agar tidak terjadi asumsi yang salah tentang sisi input dan target.



Gambar 3.1 Contoh pasangan label map dan foto asli pada dataset CMP Facades

3.2 Preprocessing dan Augmentasi

Preprocessing diimplementasikan pada kelas `Pix2PixDataset` di `src/dataset.py`. Langkah yang dilakukan berbeda untuk data latih dan data validasi/test.

Split	Langkah
train	Pisahkan gambar menjadi input dan target; resize keduanya ke 286 × 286; random crop ke 256 × 256; random horizontal flip (peluang 50%); ubah ke tensor dan normalisasi ke [-1, 1]
val dan test	Pisahkan gambar; resize langsung ke 256 × 256; ubah ke tensor dan normalisasi ke [-1, 1]; tanpa augmentasi

Posisi crop dan keputusan flip dibuat identik untuk input dan target. Jika input di-flip tetapi target tidak, pasangan gambar menjadi rusak dan model belajar hal yang salah tanpa memunculkan pesan error apa pun. Normalisasi ke rentang [-1, 1] dilakukan agar skala data sama dengan keluaran generator yang berakhir dengan Tanh, sehingga loss L1 membandingkan dua nilai yang skalanya setara:

`x = x / 255.0 * 2.0 - 1.0` (piksel [0, 255] → tensor [-1, 1])

`x = (x + 1.0) / 2.0` (tensor [-1, 1] → [0, 1], untuk menyimpan atau menampilkan gambar)

Format tensor yang disepakati pada kontrak teknis:

Titik	Shape	Rentang nilai
Satu item dari Dataset	`input` dan `target`: [3, 256, 256]	[-1, 1]
Satu batch dari DataLoader	`input` dan `target`: [B, 3, 256, 256]	[-1, 1]
Output Generator	[B, 3, 256, 256]	[-1, 1] (Tanh)
Input Discriminator	[B, 6, 256, 256]	[-1, 1]
Output Discriminator	[B, 1, 30, 30]	logits (tanpa sigmoid)

Hasil pengecekan shape dan rentang nilai (smoke test dataset): setiap item input dan target berukuran [3, 256, 256] bertipe float32 dengan nilai pada rentang [-1, 1] di ketiga split (train 400, val 100, test 106 file), dan satu batch dari DataLoader berukuran [2, 3, 256, 256]. Pasangan input dan target tetap sejajar setelah augmentasi, dicek secara visual pada dataset_check.png (kolom kiri label map, kolom kanan foto).

3.3 Arsitektur Model

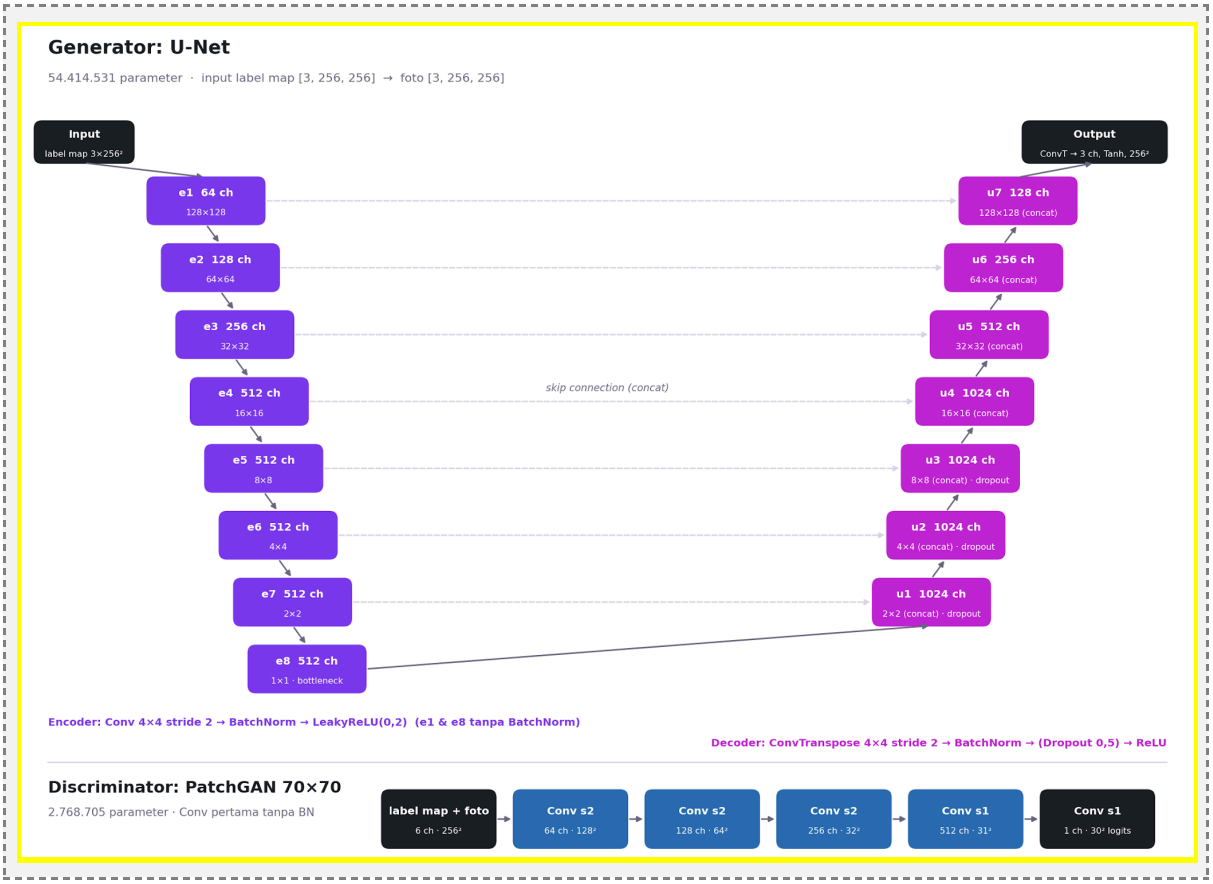
Generator: U-Net (`src/models/generator.py`). Encoder terdiri atas 8 blok downsampling berupa Conv 4×4 stride 2, BatchNorm, dan LeakyReLU (0,2), sehingga ukuran spasial turun 256 → 128 → … → 1 dengan jumlah channel 64, 128, 256, 512, 512, 512, 512, 512. Blok encoder pertama dan bottleneck (1×1) tidak memakai BatchNorm. Decoder terdiri atas 7 blok upsampling berupa ConvTranspose 4×4 stride 2, BatchNorm, Dropout 0,5 (pada 3 blok pertama), dan ReLU. Output tiap blok decoder digabung (concat) dengan output encoder yang ukurannya sama sebagai skip connection. Lapisan akhir berupa ConvTranspose ke 3 channel dengan Tanh, sehingga keluaran berada pada rentang [-1, 1], sama dengan rentang data.

Skip connection penting untuk tugas ini karena struktur pada label map (posisi jendela, pintu, balkon) harus muncul di posisi yang sama pada foto hasil. Tanpa skip connection, informasi posisi harus melewati bottleneck 1×1 dan banyak yang hilang.

Discriminator: PatchGAN 70×70 (`src/models/discriminator.py`). Input berupa label map dan gambar (asli atau palsu) yang digabung menjadi 6 channel; penggabungan dilakukan di dalam discriminator. Arsitekturnya 4 blok Conv 4×4 (stride 2, 2, 2, 1) dengan channel 64 → 128 → 256 → 512, lalu Conv akhir ke 1 channel; blok pertama tanpa BatchNorm. Keluarannya grid 30×30 logits, tiap nilainya menilai satu patch gambar sekitar 70×70 piksel. Tidak ada sigmoid di akhir, karena sigmoid sudah termasuk di dalam `BCEWithLogitsLoss`. Dengan menilai per patch, discriminator berfokus pada tekstur dan detail lokal, sedangkan struktur global dijaga oleh loss L1.

Inisialisasi bobot: Conv dan ConvTranspose ~ Normal(0; 0,02), BatchNorm weight ~ Normal(1; 0,02), dan bias = 0.

Komponen	Struktur ringkas	Jumlah parameter
Generator U-Net	Encoder 8 blok, decoder 7 blok dengan skip connection, Tanh di akhir	54.414.531
Discriminator PatchGAN 70×70	Input 6 channel, 4 blok Conv + Conv akhir, output [B, 1, 30, 30], tanpa sigmoid	2.768.705



Gambar 3.2 Arsitektur generator U-Net dan discriminator PatchGAN

3.4 Fungsi Loss dan Training

Fungsi loss diimplementasikan pada kelas `Pix2PixLoss` di `src/losses.py`.

```
L_D = 0,5 × [ BCE(D(x, y), 1) + BCE(D(x, G(x)), 0) ]
L_G = λ_adv × BCE(D(x, G(x)), 1) + λ_L1 × |G(x) − y|
```

Di sini x adalah label map, y adalah foto asli, dan G(x) adalah foto hasil generator. Nilai λ_adv = 1 dan λ_L1 = 100. Faktor 0,5 pada loss discriminator memperlambat pembelajaran discriminator supaya tidak terlalu cepat mengalahkan generator. Suku adversarial membuat hasil tampak seperti foto asli (tajam), sedangkan L1 menjaga hasil tetap dekat dengan target (struktur dan warna secara umum).

Satu iterasi training (`src/train.py`) berjalan dengan urutan berikut:

1. Generator membuat gambar palsu dari label map.
2. Update discriminator dengan pasangan asli dan pasangan palsu. Gambar palsu di-`detach()` agar gradien discriminator tidak mengalir ke generator.
3. Bekukan discriminator, lalu update generator memakai penilaian discriminator dan loss L1.

Hyperparameter baseline:

Hyperparameter	Nilai
Optimizer	Adam, β1 = 0,5, β2 = 0,999
Learning rate	2×10^{-4} konstan sampai epoch 100, lalu turun linear menuju 0 sampai epoch 200
Batch size	1
λ_L1 / λ_adv	100 / 1
Epoch	200 (400 iterasi per epoch, total 80.000 iterasi)
Seed	42

Checkpoint dan resume. File `last.pt` disimpan tiap epoch dan memuat semua yang dibutuhkan untuk melanjutkan training secara persis (bobot, state optimizer, scheduler, dan posisi random number generator). Penyimpanan dilakukan secara atomik (ditulis ke file sementara lalu di-rename), sehingga jika PC mati saat menyimpan, checkpoint sebelumnya tetap utuh. File `best.pt` (val L1 terendah) dan `epoch_050/100/150/200.pt` hanya berisi bobot untuk keperluan evaluasi.

Metrik selama training. Setiap epoch dihitung L1 pada split val (`val_L1`). Loss GAN naik-turun dan sulit dibaca, sehingga `val_L1` dipakai sebagai indikator yang lebih stabil.

3.5 Lingkungan Eksperimen

Item	Spesifikasi
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB VRAM (arsitektur Blackwell, compute capability sm_120)
Sistem operasi	Windows 11
Python	3.14.6
PyTorch	≥ 2.7, build CUDA 12.8 (cu128)
Seed	42 untuk semua eksperimen
Waktu training	sekitar 39 menit untuk 200 epoch pada run baseline (sekitar 12 detik per epoch)

Semua run yang masuk laporan dijalankan pada PC yang sama (milik Zafira), sehingga waktu training dan hasil antar eksperimen dapat dibandingkan secara adil. Pada awalnya training direncanakan di Google Colab (GPU T4), kemudian dipindahkan ke PC lokal melalui pembaruan kontrak teknis versi 1.2.

3.6 Alur Kerja Tim dan Git

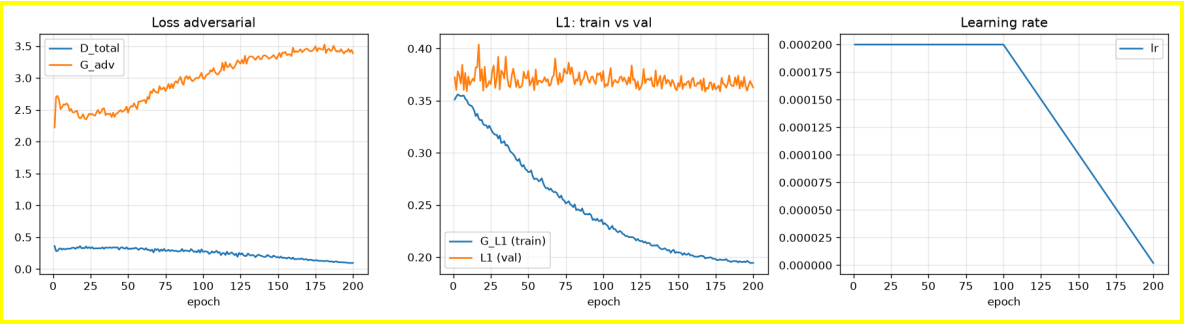
- Satu branch per bagian kerja: `feat/model` dan `feat/training` (Zafira), `feat/dataset` (Fira), `docs/laporan` (Ce).
- Tidak ada push langsung ke `main`. Semua perubahan masuk lewat Pull Request yang direview minimal satu anggota lain (review silang).
- Urutan merge: dataset dan model lebih dulu, kemudian training, karena training membutuhkan keduanya.
- Setiap anggota menjalankan smoke test (cek shape dan rentang nilai) untuk file miliknya sebelum membuat Pull Request.
- Format pesan commit: `feat(dataset): ...`, `fix(model): ...`, `docs(log): ...`, `exp(train): ...`.

Ringkasan aktivitas repositori: 56 commit di semua branch, 46 di antaranya bukan commit merge (Zafira 38, Ce 7, Fira 1), serta 9 Pull Request yang di-merge ke main.

BAB 4. EKSPERIMEN DAN HASIL

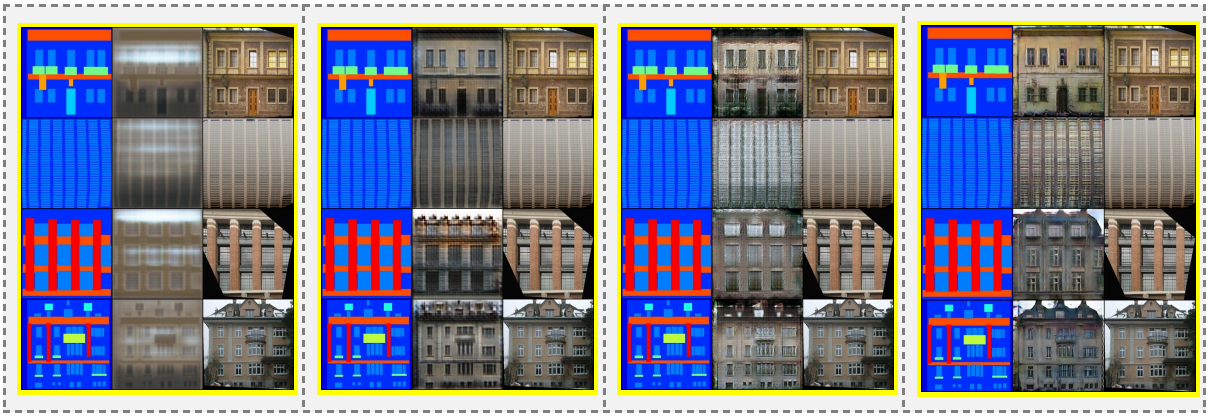
Seluruh angka pada bab ini bersumber dari `docs/hasil/results_table.csv`, yaitu hasil `evaluate.py` pada split test.

4.1 Hasil Training Baseline



Gambar 4.1 Grafik loss selama training run baseline (200 epoch)

Untuk melihat perkembangan hasil selama training, pada setiap epoch dibuat sampel dari 4 gambar validasi yang sama. Gambar berikut berasal dari run `bs4`; tiap baris menampilkan label map | hasil generator | foto asli.



Gambar 4.2 Perkembangan hasil generator pada run bs4 (epoch 1, 10, 50, dan 200)

- **Epoch 1:** hasil masih berupa bayangan abu-abu yang buram. Posisi lantai dan jendela baru terlihat samar.
- **Epoch 10:** struktur bangunan sudah jelas (baris jendela, garis lantai), tetapi warna masih gelap dan banyak bercak.
- **Epoch 50:** tekstur mulai muncul (kusen, bata, ornamen), namun masih kasar dan ada noise kehijauan.
- **Epoch 200:** hasil paling bersih, warna dinding lebih masuk akal, dan detail jendela lebih rapi. Penurunan learning rate pada epoch 101–200 membantu menghaluskan hasil.

Pengamatan pada proses training:

- Training stabil sampai epoch 200, tanpa *mode collapse* (generator tidak menghasilkan gambar yang sama untuk semua input) dan tanpa loss yang meledak.
- `val_L1` sudah hampir datar sejak sekitar epoch 25 (berkisar 0,36–0,38). Setelah itu peningkatan kualitas terlihat pada ketajaman dan detail gambar, bukan pada L1.
- `D_total` terus turun, terutama setelah epoch 100 ketika learning rate mulai diturunkan, hingga sekitar 0,10 di akhir. Artinya discriminator makin unggul atas generator. Hal ini umum pada Pix2Pix dengan dataset kecil, dan hasil visualnya tetap baik.
- Waktu training sekitar 39 menit (sekitar 12 detik per epoch) di RTX 5060 Ti.

4.2 Eksperimen Tuning

Lima run dilatih dengan konfigurasi yang sama (200 epoch, seed 42) kecuali satu hyperparameter yang diubah. Evaluasi memakai checkpoint epoch terakhir pada split test (106 gambar).

Run	Yang diubah	L1 ↓	PSNR (dB) ↑	SSIM ↑	Ketajaman †	D_total akhir	Waktu (menit)
`baseline`	–	0,3699	12,59	0,2251	0,513	0,096	39
`l1_only`	$\lambda_{adv} = 0$	**0,3576**	**12,95**	**0,3178**	0,039	0,000	38
`lambda10`	$\lambda_{L1} = 10$	0,3768	12,43	0,2230	0,501	0,243	39
`lr1e-4`	$lr = 1\times10^{-4}$	0,3736	12,49	0,2281	0,477	0,110	38
`bs4`	batch size = 4	0,3619	12,55	0,2470	**0,569**	0,173	**16**

† Ketajaman adalah rasio *variance of Laplacian* gambar hasil terhadap foto asli. Nilai 1 berarti setajam foto asli, dan nilai jauh di bawah 1 berarti buram. Angka tebal menunjukkan nilai terbaik pada kolomnya.



Gambar 4.3 Grafik loss gabungan kelima run tuning

l1_only (tanpa loss adversarial). Run ini unggul pada semua metrik kemiripan (L1, PSNR, SSIM), tetapi ketajamannya hanya 0,039, sekitar 13 kali lebih buram dibanding run lain. Tanpa discriminator, generator belajar menghasilkan "rata-rata" dari semua kemungkinan foto, yang aman bagi L1 tetapi buram. `D_total` langsung turun ke 0 karena discriminator tetap dilatih sementara generator tidak pernah berusaha menipunya. Grafik

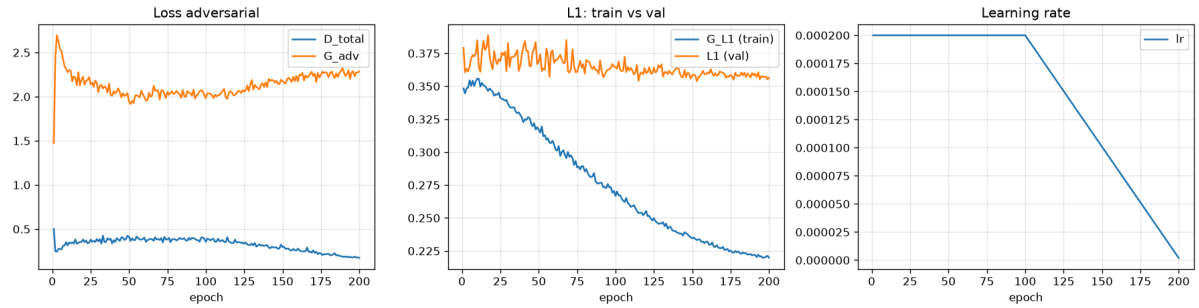
`val_L1` pada run ini juga menunjukkan indikasi overfitting ringan: nilai terendah sekitar 0,33 pada epoch 10–15, lalu naik ke sekitar 0,35.

Hasil ini menunjukkan dua hal. Pertama, loss adversarial adalah komponen yang membuat hasil Pix2Pix tajam, sesuai klaim Isola dkk. (2017). Kedua, L1, PSNR, dan SSIM saja tidak cukup untuk menilai model generatif: model yang hasilnya paling buram justru mendapat skor kemiripan terbaik.

lambda10 (bobot L1 diturunkan dari 100 ke 10). Detail yang muncul paling banyak, tetapi artefak warna juga paling banyak (bercak warna-warni yang tidak berhubungan dengan label map). Dengan bobot L1 kecil, generator lebih bebas mengikuti discriminator dan lebih sering "mengarang". Metrik kemiripannya terburuk, `D_total` paling tinggi, dan `val_L1` paling berfluktuasi.

lr1e-4 (learning rate setengah dari baseline). Hasilnya mirip baseline dengan detail sedikit lebih sedikit, dan semua metrik sedikit lebih buruk. Dengan jumlah epoch yang sama, learning rate yang lebih kecil membuat model belajar lebih lambat.

bs4 (batch size 4). Ini run GAN terbaik: L1, SSIM, dan ketajaman terbaik di antara run yang memakai loss adversarial, serta secara visual paling bersih dengan warna yang lebih bervariasi. Waktu training 2,4 kali lebih cepat karena GPU memproses 4 gambar sekaligus. Menariknya, jumlah update hanya 20.000 (seperempat dari baseline) tetapi hasilnya justru lebih baik. Kemungkinan penyebabnya adalah gradien dan statistik BatchNorm yang dihitung dari 4 gambar lebih stabil dibanding dari 1 gambar.

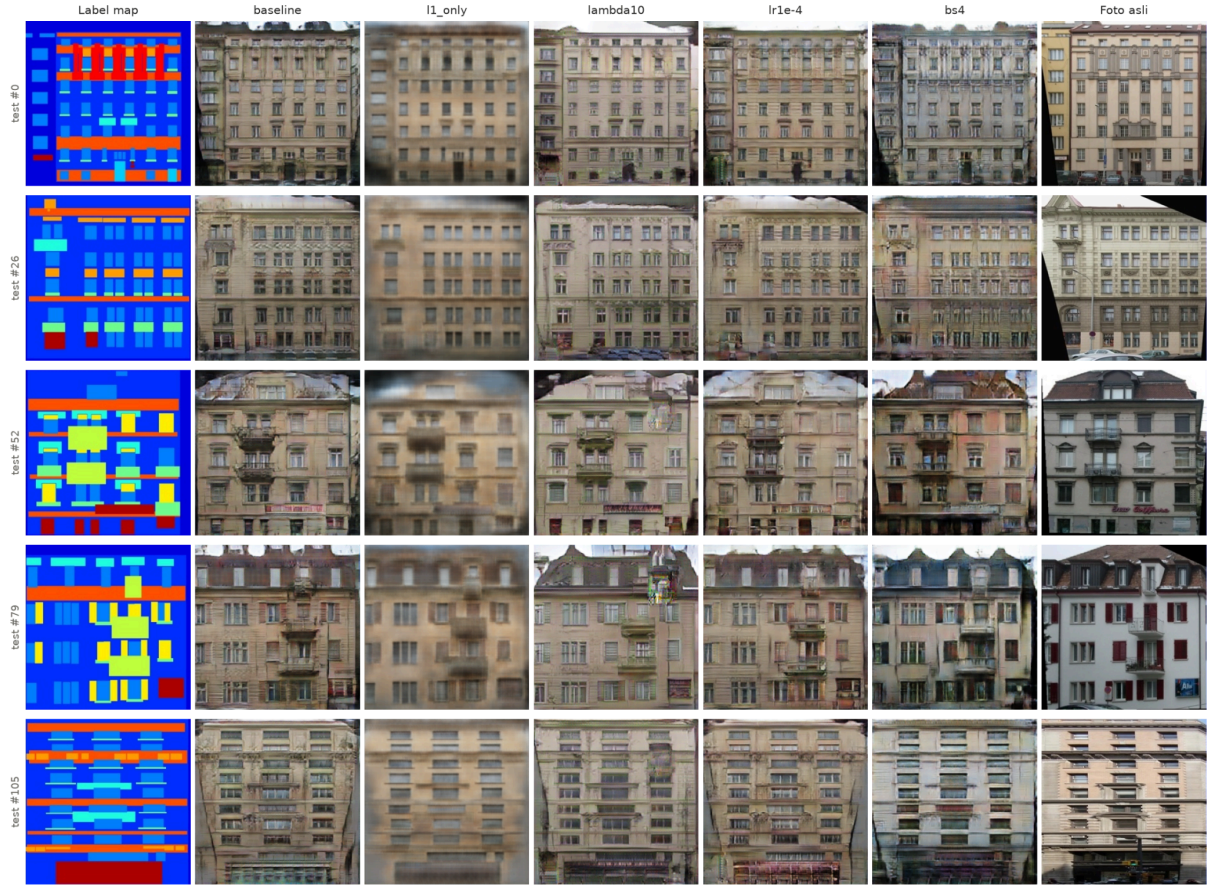


Gambar 4.4 Grafik loss selama training run bs4

Kesimpulan tuning: konfigurasi terbaik adalah `bs4`.

Keterbatasan: setiap konfigurasi hanya dijalankan dengan satu seed, dan selisih metrik antar run GAN kecil (L1 berkisar 0,362–0,377). Sebagian selisih ini dapat disebabkan oleh faktor acak. Untuk kesimpulan yang lebih kuat, setiap konfigurasi perlu dijalankan dengan beberapa seed.

4.3 Hasil Visual

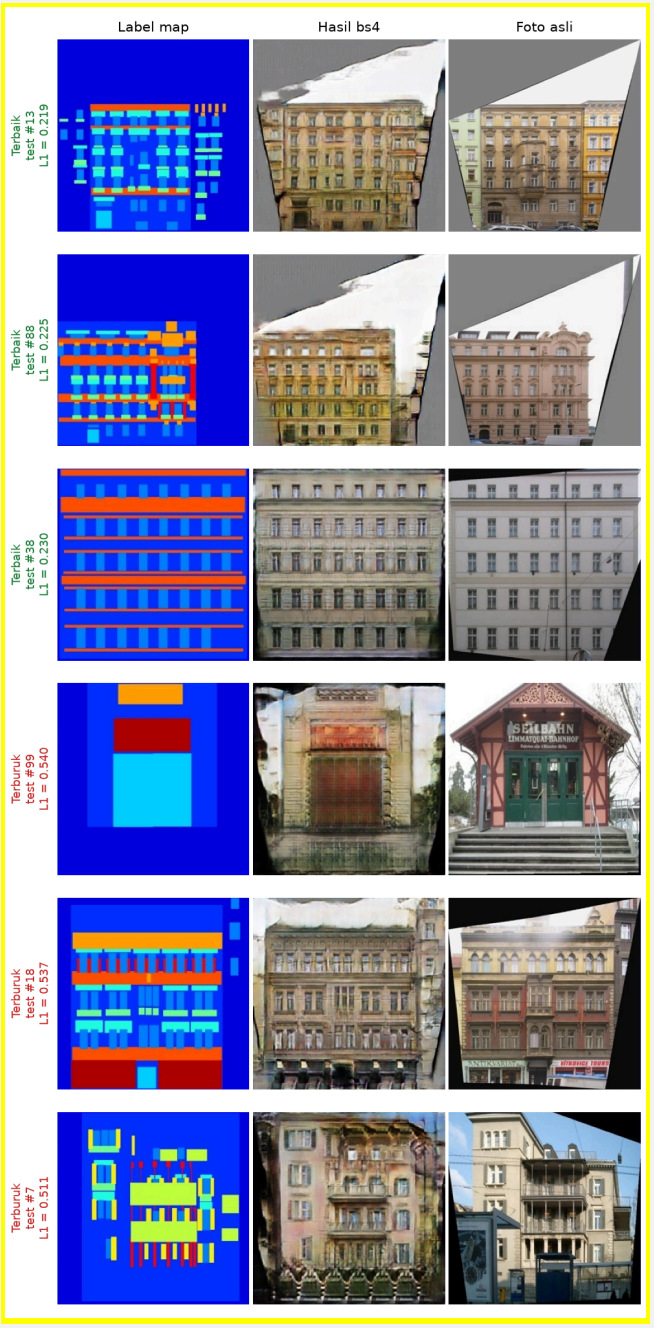


Gambar 4.5 Perbandingan hasil semua run. Kolom: label map | baseline | l1_only | lambda10 | lr1e-4 | bs4 | foto asli

- **Semua run berhasil mengikuti struktur label map.** Posisi jendela, balkon, pintu, dan lantai sesuai dengan input.
- **l1_only** buram seperti foto yang tidak fokus, dengan warna cokelat yang rata.
- **Run dengan GAN** menghasilkan tekstur yang meyakinkan (ornamen, kusen jendela, bata), walaupun detailnya tidak sama dengan foto asli.
- **Warna hasil berbeda dari foto asli.** Ini bukan kesalahan model: label map tidak berisi informasi warna, sehingga satu label map cocok dengan banyak foto yang sama-sama masuk akal (gedung krem, abu, atau putih). Hal ini juga menjelaskan mengapa PSNR rendah (sekitar 12–13 dB), karena L1 dan PSNR menghukum warna yang masuk akal tetapi berbeda dari foto asli.
- **Kegagalan yang sering muncul:** bercak putih atau abu di area langit bagian atas gambar (baseline, lr1e-4), artefak warna-warni (lambda10, contoh test #52 dan #79), dan garis vertikal (bs4, test #79). Area langit dan tepi gambar paling sering gagal karena pada label map area tersebut hanya berupa latar belakang tanpa struktur.

Gambar 4.6 menampilkan tiga hasil dengan L1 terkecil dan tiga hasil dengan L1 terbesar pada run bs4 (rata-rata L1 di split test 0,362). Hasil terbaik (L1 0,219–0,230) adalah

fasad dengan pola jendela yang teratur dan warna netral, serta foto yang memiliki area latar abu-abu atau putih yang luas (test #13 dan #88). Area latar seperti ini mudah ditiru generator, sehingga sebagian skor baiknya berasal dari latar, bukan dari fasadnya. Hasil terburuk (L1 0,511–0,540) muncul pada tiga kasus: bangunan yang tidak mirip fasad gedung pada umumnya (test #99, stasiun kecil berdinding kayu dengan pintu hijau dan tangga), sehingga generator menggambar fasad klasik yang sama sekali berbeda; warna asli yang jauh dari warna umum dataset (test #18, gedung merah dan kuning dengan pojok hitam), padahal struktur jendelanya sudah benar; serta foto yang memuat objek di depan bangunan seperti halte, pohon, dan kabel (test #7) yang tidak ada di label map. Kasus test #18 menunjukkan bahwa L1 lebih banyak menilai kecocokan warna daripada ketepatan struktur.



Gambar 4.6 Contoh hasil terbaik dan terburuk pada split test

4.4 Evaluasi Kuantitatif

Semua metrik dihitung dengan kode sendiri di `src/metrics.py` (tanpa library tambahan) untuk 106 gambar test.

Metrik	Skala	Arti
L1	piksel [-1, 1]	Rata-rata selisih absolut piksel, sama dengan `val_L1` pada training
PSNR	piksel [0, 1], dB	10·log10(1 / MSE); makin tinggi makin mirip
SSIM	[0, 1]	Kemiripan struktur lokal (kecerahan, kontras, struktur) dengan jendela Gaussian 11×11
Ketajaman	rasio	*Variance of Laplacian* hasil dibagi foto asli; mengukur tingkat blur

Metrik dibaca dalam dua kelompok. Kelompok pertama adalah kemiripan dengan foto asli (L1, PSNR, SSIM); model yang "bermain aman" dengan hasil buram cenderung unggul di sini. Kelompok kedua adalah ketajaman, yang menangkap blur yang tidak terlihat pada metrik kemiripan. Kasus `l1_only` menunjukkan pentingnya membaca keduanya bersama: skor kemiripannya terbaik, tetapi ketajamannya terburuk.

Sebagai pembanding, PSNR sekitar 12–13 dB dan SSIM sekitar 0,22–0,32 adalah kisaran yang wajar untuk dataset Facades, karena hubungan label map → foto bersifat satu-ke-banyak.

BAB 5. LOG INTERAKSI LLM DAN PENANGANAN ERROR

5.1 Ringkasan Sesi

Setiap anggota mencatat sesi interaksinya dengan LLM pada folder `docs/log_interaksi/` dengan format nama `YYYY-MM-DD_<peran>_<topik>.md`. Daftar lengkap sesi dan tautan chat tersedia pada `DAFTAR_SESI.md`. Ringkasannya sebagai berikut.

No	Tanggal	Anggota	Modul	Tautan chat
1	13/10/2026	Zafira	Kontrak teknis dan Generator U-Net	https://claude.ai/share/74d5320c-7fd1-4007-a087-5b73296bf1fb https://claude.ai/share/a98da975-f146-4377-9c2b-ccfb73e7d4fc
2	14/10/2026	Zafira	Discriminator PatchGAN dan fungsi loss	https://claude.ai/share/8f52eca0-7534-4e7b-bff0-fee2f6ed0f58
3	15/10/2026	Fira	Dataset Pix2Pix	https://claude.ai/share/8548386e-79b6-415b-a6e2-170cd8d9d1df
4	15/10/2026	Zafira	Training loop	https://claude.ai/share/57f605fb-9770-477e-841d-1e2d39a075c5
5	25/10/2026	Zafira	Lanjutan Tuning dan evaluasi	https://claude.ai/share/76a63b11-5388-4948-939c-83bc384ed692
6	13/10/2026	Ce	Dokumentasi: kerangka laporan, kompilasi hasil dan log	https://claude.ai/share/6fb00385-30af-43ea-a6c2-41b35085de79

5.2 Rekap Error dan Solusi

Error dan kendala yang muncul selama pengembangan dikonsultasikan kepada LLM, dianalisis, lalu dicatat pada log. Pesan error utuh, prompt perbaikan, dan analisis lengkap tersedia pada file log masing-masing. Ringkasannya:

No	Masalah	Solusi	Sumber log
1	File baseline_train.log kosong meskipun proses training sudah berjalan lama. Hal ini terjadi karena output print Python tertahan di buffer sehingga belum langsung ditulis ke file.	Menambahkan set PYTHONUNBUFFERED=1 pada run_experiments.bat agar output log ditulis secara langsung dan dapat dipantau selama training berlangsung.	- **Tanggal Sesi:** 16–26/10/2026 - **LLM:** Claude (claude.ai, proyek "Pix2Pix Gen AI"), lanjutan chat training loop - **Tautan Sesi Chat:** https://claude.ai/share/76a63b11-5388-4948-939c-83bc384ed692 - outputs/_batch_logs/baseline_train.log
2	PC mengalami mati mendadak ketika proses batch training sedang berjalan sehingga status penyelesaian eksperimen perlu diperiksa kembali.	Memeriksa epoch terakhir pada file losses.csv dan keberadaan metrics.json untuk memastikan status setiap eksperimen. Hasil pemeriksaan menunjukkan bahwa kelima run telah selesai sebelum PC mati.	- **Tanggal Sesi:** 16–26/10/2026 - **LLM:** Claude (claude.ai, proyek "Pix2Pix Gen AI"), lanjutan chat training loop - **Tautan Sesi Chat:** https://claude.ai/share/76a63b11-5388-4948-939c-83bc384ed692 - losses.csv dan metrics.json

No	Masalah	Solusi	Sumber log
3	Saat menyimpan sampel gambar pada training di GPU muncul RuntimeError: Expected all tensors to be on the same device. Tensor target masih di CPU sementara hasil generator di GPU, sehingga torch.stack gagal.	Ketiga tensor dipindahkan ke CPU sebelum digabung di save_sample_grid (src/utils.py). Bug ini tidak tertangkap saat LLM menguji kode karena lingkungannya tanpa GPU, sehingga dummy run singkat di GPU dijadikan langkah wajib sebelum training panjang.	- **Tanggal Sesi:** 15–16/10/2026 - **LLM:** Claude (claude.ai, proyek "Pix2Pix Gen AI"), chat training loop - **Tautan Sesi Chat:** https://claude.ai/share/76a63b11-5388-4948-939c-83bc384ed692 - 2026-10-15_training_train-loop.md
4	Saat kode disisir ulang untuk mencari bug serupa, ditemukan bahwa load_checkpoint memuat checkpoint langsung ke GPU. RNG state ikut pindah ke GPU, padahal torch.set_rng_state hanya menerima tensor CPU, sehingga resume pertama akan crash.	load_checkpoint selalu memakai map_location="cpu"; bobot model dan state optimizer tetap berakhir di GPU melalui load_state_dict(). Resume kemudian diuji di GPU dan berhasil melanjutkan training.	- **Tanggal Sesi:** 15–16/10/2026 - **LLM:** Claude (claude.ai, proyek "Pix2Pix Gen AI"), chat training loop - **Tautan Sesi Chat:** https://claude.ai/share/76a63b11-5388-4948-939c-83bc384ed692 - 2026-10-15_training_train-loop.md

Pelajaran utama dari penanganan error: Proses debugging menggunakan LLM akan lebih efektif apabila prompt menyertakan pesan error secara lengkap, konteks kode yang bermasalah, serta kondisi saat error terjadi. Selain itu, setiap perbaikan perlu diuji terlebih dahulu melalui smoke test untuk memastikan fungsi berjalan sesuai harapan sebelum menjalankan training dalam durasi panjang. Pemeriksaan log secara berkala juga penting untuk memantau progres training dan memastikan setiap eksperimen selesai dengan baik.

5.3 Pola Prompting yang Efektif

Kelompok memakai pendekatan kontrak lebih dulu (*contract-first*): keputusan arsitektur, format tensor, dan signature kelas disepakati dalam `KONTRAK.md` sebelum LLM diminta membuat kode, sehingga setiap anggota dapat memberi prompt yang spesifik dan

hasilnya tetap kompatibel saat digabung. Variasi prompt yang dipakai dikelompokkan pada tiga tahap:

Tahap	Contoh prompt	Mengapa efektif
Inisiasi (kode awal)	Sesuai KONTRAK.md, aku mulai bagian model: Generator U-Net di `src/models/generator.py`. Sebelum nulis kode, jelaskan dulu arsitektur U-Net Pix2Pix: susunan encoder dan decoder-nya, kenapa butuh skip connection, dan di layer mana dipakai dropout. Habis itu baru kita bikin kodenya bagian per bagian.	Prompt disusun secara bertahap dengan mengacu pada kontrak teknis (KONTRAK.md) dan menentukan file yang akan dikerjakan. Meminta penjelasan arsitektur sebelum penulisan kode membantu memastikan pemahaman terhadap komponen U-Net, skip connection, dan dropout. Pembagian implementasi per bagian juga memudahkan proses pengecekan dan mengurangi risiko ketidaksesuaian antar-modul.
Debugging	Di PC (GPU) muncul error waktu nyimpen sampel di epoch 1: [traceback: RuntimeError: Expected all tensors to be on the same device, but got tensors is on cpu, different from other tensors on cuda:0]. Kenapa ini nggak ketahuan waktu kamu tes? Tolong perbaiki.	Prompt menyertakan traceback lengkap dan kondisi saat error terjadi (training di GPU, epoch 1, saat menyimpan sampel), sehingga LLM langsung menemukan penyebabnya: tensor target masih di CPU sementara hasil generator di GPU. Pertanyaan "kenapa ini nggak ketahuan" membuat LLM menjelaskan bahwa pengujiannya sendiri berjalan tanpa GPU, lalu menyisir ulang bagian kode yang hanya berjalan di GPU. Dari situ ditemukan bug kedua pada load_checkpoint sebelum training panjang dimulai.

Tahap	Contoh prompt	Mengapa efektif
Optimasi	Iya, tolong buatkan evaluate.py, script tabel + gambar perbandingan + grafik gabungan, dan file .bat buat jalanin kelima run. Ukuran last.pt dan best.pt dari dummy run masing-masing 655 MB. Sisa disk C: 91 GB dari 952 GB. Sesuaikan save_every-nya, aku pikir 50 cukup.	Prompt menyertakan informasi teknis yang spesifik, seperti ukuran checkpoint, jumlah run, target epoch, dan kapasitas penyimpanan yang tersedia. Informasi ini membantu LLM mempertimbangkan kebutuhan penyimpanan sebelum menentukan frekuensi penyimpanan checkpoint. Dengan demikian, optimasi tidak hanya berfokus pada pembuatan fitur evaluasi, tetapi juga mempertimbangkan keterbatasan sumber daya perangkat.

BAB 6. KONTRIBUSI DAN KERJA SAMA TIM

6.1 Kontribusi Anggota

Anggota	Kontribusi	Bukti
Zafira	Kontrak teknis; Generator U-Net dan Discriminator PatchGAN; fungsi loss; training loop, checkpoint, dan resume; kode evaluasi (metrik, evaluate, compare_runs); lima run tuning; draf hasil dan analisis	Branch `feat/model` dan `feat/training`; PR #1, PR #3, PR #5, dan PR #6
Fira	Pencarian dan persiapan dataset; pemisahan pasangan gambar; resize, normalisasi, dan augmentasi; Dataset dan DataLoader; visualisasi input → output → target	Branch `feat/dataset` PR #4
Ce	Kerangka laporan; template dan kompilasi log interaksi; review dan merge PR; penyusunan laporan akhir dan bahan presentasi	Branch `docs/laporan` PR #2

6.2 Cara Integrasi

Integrasi pekerjaan tiga anggota dijaga melalui kontrak teknis (`KONTRAK.md`) yang memuat keputusan bersama, signature kelas, dan format tensor. Perubahan kontrak hanya dilakukan lewat Pull Request setelah didiskusikan di grup. Selain itu, kelompok menerapkan review silang agar semua anggota memahami kode anggota lain (penting untuk presentasi) dan menjalankan smoke test sebelum membuat Pull Request.

Riwayat perubahan kontrak: versi 1.0 (kesepakatan awal), versi 1.1 (pembagian peran diperbarui), dan versi 1.2 (lingkungan training dipindahkan dari Google Colab T4 ke PC lokal dengan RTX 5060 Ti, dan notebook menjadi opsional).

6.3 Kendala Kerja Sama dan Cara Mengatasinya

- Perubahan lingkungan training dari Colab ke PC lokal diselesaikan dengan memperbarui kontrak (v1.2) dan menambahkan aturan khusus untuk PyTorch CUDA 12.8 serta Windows.
- Hanya satu anggota (Zafira) yang memiliki akun Claude. Sesi chat dibuat di akun Zafira lalu dibagikan lewat tautan share, dan anggota lain melanjutkan pekerjaannya berdasarkan isi sesi tersebut.
- Kode ditulis di laptop, sedangkan training dijalankan di PC. Setiap perubahan harus di-push dari laptop lalu di-pull di PC, sehingga kelompok membiasakan commit kecil dan smoke test di PC setelah setiap pull.
- Push pertama Fira dari laptop Mac sempat tertahan karena Git menunggu login GitHub yang tidak terlihat di terminal. Push berhasil setelah proses login GitHub diselesaikan.
- Saat review, anggota sempat membuka Pull Request yang salah (PR #2, padahal yang dimaksud PR #3). Untuk mencegahnya, tautan PR dikirim langsung di grup.
- Push beberapa kali ditolak karena branch di GitHub sudah berisi commit yang dibuat dari perangkat lain. Masalah ini diselesaikan dengan git pull --rebase sebelum push.

BAB 7. KESIMPULAN DAN SARAN

7.1 Kesimpulan

1. Pix2Pix berhasil dibangun dari nol dengan PyTorch melalui vibe coding: generator U-Net (54.414.531 parameter) dan discriminator PatchGAN 70×70 (2.768.705 parameter). Training berjalan stabil selama 200 epoch tanpa mode collapse.
2. Semua konfigurasi mampu mengikuti struktur label map (posisi jendela, pintu, balkon, dan lantai), dengan tekstur yang meyakinkan pada run yang memakai loss adversarial.
3. Loss adversarial berperan penting terhadap ketajaman: run tanpa loss adversarial (`l1_only`) memiliki skor kemiripan terbaik tetapi hasilnya sangat buram (ketajaman 0,039). Karena itu L1, PSNR, dan SSIM harus dibaca bersama ukuran ketajaman dan pengamatan visual.
4. Konfigurasi terbaik adalah batch size 4 (`bs4`): L1, SSIM, dan ketajaman terbaik di antara model GAN, secara visual paling bersih, dan 2,4 kali lebih cepat dilatih (16 menit dibanding sekitar 39 menit).
5. Kontrak teknis, review silang, dan log interaksi membantu kelompok menggabungkan pekerjaan tiga anggota dengan terdokumentasi.

7.2 Keterbatasan

- Setiap konfigurasi hanya dijalankan dengan satu seed, dan selisih antar run GAN kecil sehingga kesimpulan tuning bersifat indikatif.
- Hubungan label map → foto bersifat satu-ke-banyak (warna tidak ditentukan oleh label map), sehingga metrik kemiripan seperti PSNR dan SSIM cenderung rendah walaupun hasilnya masuk akal.
- Dataset berukuran kecil dan hanya satu dataset yang diuji.

7.3 Saran

- Menjalankan setiap konfigurasi dengan beberapa seed dan melaporkan rata-rata beserta simpangannya.
- Menambahkan metrik yang lebih sesuai untuk model generatif, misalnya metrik perseptual.
- Mencoba variasi lain, seperti batch size lebih besar, resolusi lebih tinggi, atau dataset yang lebih besar.

DAFTAR PUSTAKA

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Nets.* Advances in Neural Information Processing Systems (NeurIPS).

Isola, P., Zhu, J.-Y., Zhou, T., & Efros, A. A. (2017). *Image-to-Image Translation with Conditional Adversarial Networks.* IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation.* Medical Image Computing and Computer-Assisted Intervention (MICCAI).

Tyleček, R., & Šára, R. (2013). *Spatial Pattern Templates for Recognition of Objects with Regular Structure.* German Conference on Pattern Recognition (dataset CMP Facade).

PyTorch Documentation. https://pytorch.org/docs/

LAMPIRAN

- **Lampiran A.** Tautan seluruh sesi chat LLM (lihat `docs/log_interaksi/DAFTAR_SESI.md`).
- **Lampiran B.** Log Interaksi LLM seluruh anggota (folder `docs/log_interaksi/`).
- **Lampiran C.** Riwayat commit dan Pull Request: https://github.com/zafiranurkamila/pix2pix-scratch
- **Lampiran D.** Kontrak teknis (`KONTRAK.md`).